

محمدرضا شریفی

۴۰۱۱۰۱۹۳۲

فاز سوم

طراحی سیستم های میکروپروسسوری

برای انجام فاز سوم درس از یکی از پیاده سازی های فاز دوم یعنی `matrix_mult_transpose` استفاده میکنیم و برای این که بتوانیم چندین دستور را به طور همزمان اجرا کنیم، از تکنیک `loop unrolling` استفاده میکنیم. جهت یافتن نقطه بهینه عملکرد پردازنده، این تابع با سه ضریب مختلف (Unroll Factors) شامل ۲، ۴ و ۸ بازنویسی شده است تا بتوانیم تعداد واحدهای محاسباتی ممیز شناور (FPU) که به صورت همزمان درگیر میشوند را به حداکثر رسانده و عمق پایپ لاین را تخمین بزنیم. در نسخه اول (`Unrolled_2`) سعی شده است ۲ دستور ضرب/جمع کاملاً مستقل تولید و به صورت همزمان وارد مسیر اجرایی (پایپ لاین) پردازنده شود. این منطق در نسخه های دوم و سوم نیز بسط داده شده تا به ترتیب ۴ و ۸ دستور مستقل به صورت موازی به سمت پردازنده ارسال گردند.

```
// Phase 3: Loop Unrolling Factor 2
void matrix_mult_unrolled_2(int n, double* a, double* b, double* c) {
    register int i, j, k;

    double* b_t = (double*)_aligned_malloc(n * n * sizeof(double), 64);
    if (b_t == NULL) return;

    register double *p_b = b;
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++, p_b++) b_t[j * n + i] = *p_b;
    }

    register double *pa, *pb_t;
    int limit = n - (n % 2);

    for (i = 0; i < n; i++, a += n) {
        for (j = 0; j < n; j++, c++) {
            register double sum1 = 0.0;
            register double sum2 = 0.0;

            pb_t = b_t + (j * n);

            for (k = 0, pa = a; k < limit; k += 2, pa += 2, pb_t += 2) {
                sum1 += pa[0] * pb_t[0];
                sum2 += pa[1] * pb_t[1];
            }

            register double total_sum = sum1 + sum2;

            for (; k < n; k++, pa++, pb_t++) {
                total_sum += (*pa) * (*pb_t);
            }

            *c = total_sum;
        }
    }
    _aligned_free(b_t);
}
```

شکل ۱: unroll factor=2

```

// Phase 3: Loop Unrolling Factor 4
void matrix_mult_unrolled_4(int n, double* a, double* b, double* c) {
    register int i, j, k;

    double* b_t = (double*)_aligned_malloc(n * n * sizeof(double), 64);
    if (b_t == NULL) return;

    register double *p_b = b;
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++, p_b++) b_t[j * n + i] = *p_b;
    }

    register double *pa, *pb_t;
    int limit = n - (n % 4);

    for (i = 0; i < n; i++, a += n) {
        for (j = 0; j < n; j++, c++) {
            register double sum1 = 0.0;
            register double sum2 = 0.0;
            register double sum3 = 0.0;
            register double sum4 = 0.0;

            pb_t = b_t + (j * n);

            for (k = 0, pa = a; k < limit; k += 4, pa += 4, pb_t += 4) {
                sum1 += pa[0] * pb_t[0];
                sum2 += pa[1] * pb_t[1];
                sum3 += pa[2] * pb_t[2];
                sum4 += pa[3] * pb_t[3];
            }

            register double total_sum = sum1 + sum2 + sum3 + sum4;

            for (; k < n; k++, pa++, pb_t++) {
                total_sum += (*pa) * (*pb_t);
            }

            *c = total_sum;
        }
    }
    _aligned_free(b_t);
}

```

شکل ۲: unroll factor=4

```

// Phase 3: Loop Unrolling Factor 8
void matrix_mult_unrolled_8(int n, double* a, double* b, double* c) {
    register int i, j, k;

    double* b_t = (double*)_aligned_malloc(n * n * sizeof(double), 64);
    if (b_t == NULL) return;

    register double *p_b = b;
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++, p_b++) b_t[j * n + i] = *p_b;
    }

    register double *pa, *pb_t;
    int limit = n - (n % 8);

    for (i = 0; i < n; i++, a += n) {
        for (j = 0; j < n; j++, c++) {
            register double sum1 = 0.0; register double sum2 = 0.0;
            register double sum3 = 0.0; register double sum4 = 0.0;
            register double sum5 = 0.0; register double sum6 = 0.0;
            register double sum7 = 0.0; register double sum8 = 0.0;

            pb_t = b_t + (j * n);

            for (k = 0, pa = a; k < limit; k += 8, pa += 8, pb_t += 8) {
                sum1 += pa[0] * pb_t[0]; sum2 += pa[1] * pb_t[1];
                sum3 += pa[2] * pb_t[2]; sum4 += pa[3] * pb_t[3];
                sum5 += pa[4] * pb_t[4]; sum6 += pa[5] * pb_t[5];
                sum7 += pa[6] * pb_t[6]; sum8 += pa[7] * pb_t[7];
            }

            register double total_sum = sum1 + sum2 + sum3 + sum4 +
                sum5 + sum6 + sum7 + sum8;

            for (; k < n; k++, pa++, pb_t++) {
                total_sum += (*pa) * (*pb_t);
            }

            *c = total_sum;
        }
    }
    _aligned_free(b_t);
}

```

شکل ۳: unroll factor=8

n	Matrix_mult_transpose	Unrolled_2	Unrolled_4	Unrolled_8	Max gain
512	0.12	0.06	0.05	0.05	2.4
1024	1	0.52	0.40	0.41	2.5
2048	8.75	5.23	4.96	5.07	1.76
4096	69	48.20	44.1	45	1.56

همان طور که مشاهده میشود به ازای unroll factor=8، دیگر گین نمیگیریم و عملکرد بهبود نمیابد و به ازای unroll factor=4 است که ماکسیمم گین را میگیریم.

N=512:

```
Do you want to run matrix_mult_unrolled_2 (y/n)? y
Execution time of matrix_mult_unrolled_2 = 0.06 s
Do you want to run matrix_mult_unrolled_4 (y/n)? y
Execution time of matrix_mult_unrolled_4 = 0.05 s
Do you want to run matrix_mult_unrolled_8 (y/n)? y
Execution time of matrix_mult_unrolled_8 = 0.05 s
```

N=1024:

```
Do you want to run matrix_mult_unrolled_2 (y/n)? y
Execution time of matrix_mult_unrolled_2 = 0.51 s
Do you want to run matrix_mult_unrolled_4 (y/n)? y
Execution time of matrix_mult_unrolled_4 = 0.39 s
Do you want to run matrix_mult_unrolled_8 (y/n)? y
Execution time of matrix_mult_unrolled_8 = 0.42 s
```

N=2048:

```
Do you want to run matrix_mult_unrolled_2 (y/n)? y
Execution time of matrix_mult_unrolled_2 = 5.20 s
Do you want to run matrix_mult_unrolled_4 (y/n)? y
Execution time of matrix_mult_unrolled_4 = 5.01 s
Do you want to run matrix_mult_unrolled_8 (y/n)? y
Execution time of matrix_mult_unrolled_8 = 5.04 s
```

N=4096:

```
Do you want to run matrix_mult_unrolled_2 (y/n)? y
Execution time of matrix_mult_unrolled_2 = 45.87 s
Do you want to run matrix_mult_unrolled_4 (y/n)? y
Execution time of matrix_mult_unrolled_4 = 43.10 s
Do you want to run matrix_mult_unrolled_8 (y/n)? y
Execution time of matrix_mult_unrolled_8 = 44.43 s
```

ماکس گین تابع matrix_mult_transpose نسبت به matrix_mult_ptr_reg برابر است با ۷.۷۸ و خود matrix_mult_ptr_reg نسبت به matrix_mult_index گین حدود ۳ برابر دارد و نهایتاً unrolled_4 نسبت به matrix_mult_transpose ماکس گین ۲.۵ برابر دارد. بنابراین ماکس گین کل تا به این جای کار برابر است با:

$$7.78 * 3 * 2.5 = 58.35$$

اجرای کد بنچمارکینگ برای اطلاع دقیق تر از ساختار پردازنده لپتاپ:

Adder Units (loop iterations is 500,000,000):

```
Avg. exec. time of ONE addition in the loop is 0.99 s
Execution time of 2 operations in the loop is 0.99 s, (0 %)
Execution time of 3 operations in the loop is 0.99 s, (0 %)
Execution time of 4 operations in the loop is 1.01 s, (3 %)
Execution time of 5 operations in the loop is 1.02 s, (4 %)
Execution time of 6 operations in the loop is 1.04 s, (5 %)
Execution time of 7 operations in the loop is 1.11 s, (13 %)
Execution time of 8 operations in the loop is 1.26 s, (27 %)
Execution time of 9 operations in the loop is 1.35 s, (36 %)
Execution time of 10 operations in the loop is 1.49 s, (51 %)
Execution time of 11 operations in the loop is 1.68 s, (70 %)
Execution time of 12 operations in the loop is 1.77 s, (80 %)
Execution time of 13 operations in the loop is 1.95 s, (98 %)
Execution time of 14 operations in the loop is 2.13 s, (116 %)
Execution time of 15 operations in the loop is 2.34 s, (137 %)
Execution time of 16 operations in the loop is 2.21 s, (124 %)
```

Multiplier Units (loop iterations is 500,000,000):

```
Avg. exec. time of ONE multiply in the loop is 1.05 s
Execution time of 2 operations in the loop is 1.04 s, (-1 %)
Execution time of 3 operations in the loop is 1.14 s, (9 %)
Execution time of 4 operations in the loop is 1.10 s, (5 %)
Execution time of 5 operations in the loop is 1.11 s, (6 %)
Execution time of 6 operations in the loop is 1.16 s, (11 %)
Execution time of 7 operations in the loop is 1.38 s, (32 %)
Execution time of 8 operations in the loop is 1.53 s, (47 %)
Execution time of 9 operations in the loop is 1.37 s, (30 %)
Execution time of 10 operations in the loop is 1.65 s, (57 %)
Execution time of 11 operations in the loop is 1.68 s, (60 %)
Execution time of 12 operations in the loop is 2.03 s, (94 %)
Execution time of 13 operations in the loop is 2.07 s, (98 %)
Execution time of 14 operations in the loop is 2.15 s, (105 %)
Execution time of 15 operations in the loop is 2.15 s, (106 %)
Execution time of 16 operations in the loop is 2.07 s, (98 %)
```

Simultaneous Adder & Multiplier Units (loop iterations is 500,000,000):

```
SIMULTANEOUS one add & one mult in the loop is 1.09 s
Execution time of 2 operations in the loop is 1.10 s, (1 %)
Execution time of 3 operations in the loop is 1.05 s, (-4 %)
Execution time of 4 operations in the loop is 1.31 s, (20 %)
Execution time of 5 operations in the loop is 1.50 s, (38 %)
Execution time of 6 operations in the loop is 1.96 s, (80 %)
Execution time of 7 operations in the loop is 2.27 s, (108 %)
Execution time of 8 operations in the loop is 2.27 s, (108 %)
```

همان طور که دیده میشود تا حدود ۶ دستور افزایش بسیار کم و ناچیزی در زمان اجرای دستور عمل دیده میشود. همچنین در تصویر پایینی مشخص است که تا ۳ جفت دستور ضرب و جمع توامان (۶ دستور عمل) زمان تفاوت خاصی نمیکند. این نشان میدهد که تا مجموعاً ۶ دستور مستقل به صورت همزمان میتوانند اجرا بشوند. بنابراین میتوان گفت که یا عمق پایپ لاین ۳ میباشد و ۲ یونیت اجرایی در هر core داریم (هم یونیت اجرایی ضرب کننده و هم ضرب کننده) و یا عمق پایپ لاین ۶ میباشد و ۱ یونیت اجرایی در هر core داریم. این کشف بنچمارک با نتیجه قسمت قبل یعنی unrolling مطابقت دارد. چرا که در آن جا نیز با ضریب ۴ دستور هنگامی که دستورات ارسال میشد بهترین گین را میگرفتیم اما با ضریب ۸ دیگر بهبود عملکرد را شاهد نبودیم. علت این است که ظرفیت کل با ۶ دستور پر میشود و ۸ از این عدد بیشتر میباشد.

به صورت کلی داریم:

*pipeline depth * number of units = capacity of total instructions*

*→ either $6 * 1 = 6$ or $3 * 2 = 6$*